HeatFX — project overview (serverless)

Repository: github.com/OrieBound/heatfx-serverless

Companion docs in repo: `SPEC.md` (detailed product & technical specification), root `README.md`, `infra/README.md`, `infra/terraform/README.md`.

Purpose of this document: Single printable / PDF-friendly overview for stakeholders, Obsidian vaults, or archival use. It consolidates what HeatFX is, what was built, how it is deployed on AWS, and where to look in source control.

Table of contents

1. Executive summary
2. Product definition
3. User-facing experience
4. Data model & persistence
5. Technical architecture
6. Repository layout
7. Implementation status
8. Configuration (environment)
9. Build & publish (frontend)
10. Deploy on AWS (summary)
11. Security & hygiene
12. Current limitations & production notes
13. References & deep links

---

1. Executive summary

HeatFX is a **serverless web application** that records **pointer activity** (mouse / touch-style interactions) on a **grid**, then visualizes the session as a **heatmap**, a **timestamp-driven replay** (not video), and a **details** view. It intentionally captures **events** (positions, clicks, drags, scroll, optional chaos-mode snapshots) rather than pixels.

The same codebase is a **reference architecture** for AWS: **static Next.js** on **S3 + CloudFront**, **API Gateway HTTP API + Lambda**, **DynamoDB**, **private S3** for payloads, **Cognito** for authentication. Infrastructure is expressed as **CloudFormation** (`infra/cloudformation/`) **or Terraform** (`infra/terraform/`) — **one IaC tool per environment**, not both against the same resources.

Status: Portfolio / reference implementation — core flows (record, results, cloud save, **My Recordings**, infra) are implemented end-to-end. Production hardening (CORS, Cognito callback URLs, IAM review, monitoring) remains the operator's responsibility.

---

2. Product definition

2.1 Problem statement

Can you replay what a user did on a page without recording a video? HeatFX answers that by storing a structured **event stream** and replaying it in the browser.

2.2 Recording duration (accurate caps)

Audience	Cap

Guests (not signed in)	**30 seconds** per session (fixed).
Signed-in users	**30, 60, 90, or 120 seconds** (user-selectable in settings). The API accepts saved sessions with `durationMs` **up to 120,000** to match the UI.

2.3 Themes (aligned with in-app `/why`)

Theme	Summary
Invisible data, made visible	Pointer events are abundant; HeatFX surfaces them as heatmap + replay without traditional page analytics embedding.
Replay without video	Event streams + timestamps; lightweight compared to screen capture.
Serverless reference	Auth, API, DB, object storage, CDN, IaC wired together.
Serverless economics	Pay-per-use; low traffic is typically inexpensive — verify current AWS pricing for your account.
Usability and polish	Intended to be engaging as well as technically interesting.
Open & self-hostable	Source and templates on GitHub; deploy into your AWS account; optional GitHub → CodePipeline → CodeBuild.

---

3. User-facing experience

3.1 Local development

```
git clone https://github.com/OrieBound/heatfx-serverless.git
cd heatfx-serverless
npm install
npm run dev
```

Open **http://localhost:3000** (or the port Next.js prints).

3.2 Recorder flow

1. **Start** recording (countdown).
2. Move, click, drag in the **grid** until **Stop** or the **session cap**.
3. **View results** — heatmap, replay, details.
4. **Save to cloud** (signed-in): persists metadata and event JSON via the API.
5. **My Recordings** (signed-in): list sessions, open one (presigned S3 for payload).

3.3 In-app documentation routes

Route	Role
`/about`	User-facing guide
`/stack`	Technical stack narrative
`/why`	Motivation and themes

4. Data model & persistence

4.1 Event types (conceptual)

Normalized coordinates **0..1** relative to the grid; `t` = milliseconds since recording start. Types include **move**, **down**, **up**, **click**, **drag_***, **scroll**, plus metadata such as drag rectangles on **drag_end**. See `types/events.ts`.

4.2 Storage layers

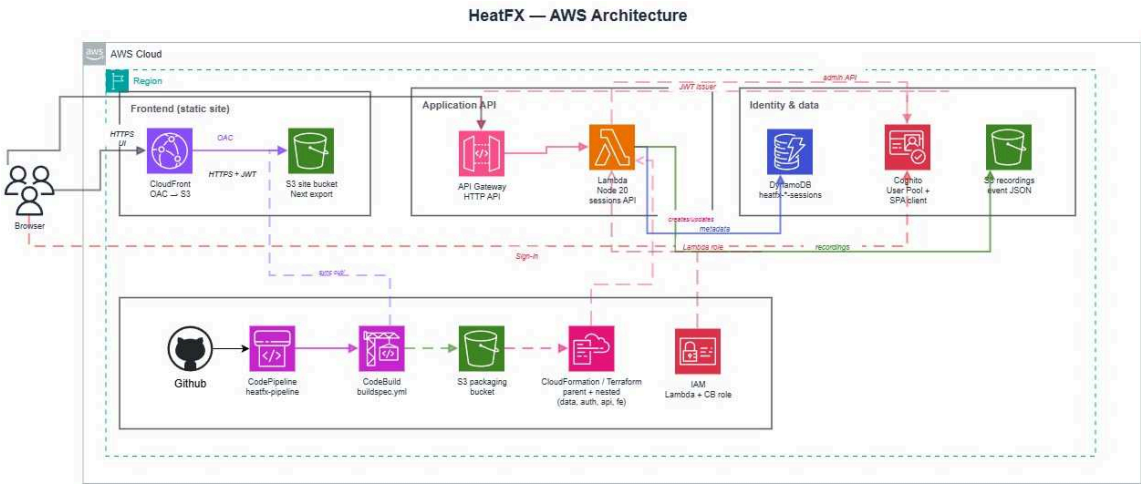
Layer	Role
DynamoDB	Session **metadata** (ids, timestamps, dimensions, counts, etc.).
S3 (private)	**Raw session JSON** (full event payload, setting snapshots, chaos data as applicable).
sessionStorage	**Same-tab handoff** only — Results page reads the payload immediately after record or after in-tab fetch from S3. **Not** the system of record.

4.3 Guests vs signed-in

- **Guests:** Record and view results in-tab; **no** cloud persistence until sign-in + save.
- **Signed-in:** Save loads the API; **My Recordings** uses API list + presigned **GET** for large JSON (avoids streaming huge bodies through Lambda).

5. Technical architecture

5.1 High-level diagram



Source: `docs/heatfx-serverless-architecture.jpg` *(embedded in root README).*

5.2 Components

Layer	Technology
Frontend	Next.js 14 (App Router), TypeScript, React 18, `output: 'export'` → static files in `out/`

CDN / site	Amazon S3 (site bucket) + **CloudFront** (OAC, private S3 origin)
API	API Gateway **HTTP API**, **Node.js 20 Lambda**
Auth	Cognito User Pool + app client; SPA uses **`amazon-cognito-identity-js`** (SRP) and custom auth pages; Hosted UI domain for redirect flows
Protected API	JWT authorizer validates Cognito tokens before protected routes
Data	DynamoDB table + private S3 bucket for recordings

No VPC in the reference design: Lambda runs in default (non-VPC) mode; services are reached over public HTTPS endpoints — typical for this serverless style.

5.3 Infrastructure as code

Path	Tooling
`infra/cloudformation/`	Parent stack + nested stacks: **data**, **auth**, **api** (SAM), **frontend**
`infra/cloudformation/pipeline/`	Optional **CodePipeline** + CodeBuild (`buildspec.yml`)
`infra/terraform/`	App stack modules mirroring the same logical architecture
`infra/terraform/pipeline/`	Optional pipeline + remote state; **`buildspec.terraform.yml`**

Rule: Do not manage the **same environment** with CloudFormation and Terraform simultaneously.

---

6. Repository layout

Area	Location
Next.js app	`app/`, `components/`, `contexts/`, `hooks/`, `types/`
CloudFormation	`infra/cloudformation/` (`parent.yaml`, `stacks/`, `nested/`)
Lambda (CFN package path)	`infra/cloudformation/api/src/`
Lambda (Terraform zip path)	`infra/terraform/api/` — keep in sync if both paths are maintained
Deploy scripts	`scripts/deploy-aws.sh`, `deploy-aws.ps1`, `deploy-pipeline.*`, `tf.sh`
CI (CFN)	`buildspec.yml`
CI (Terraform)	`buildspec.terraform.yml`
Pipeline params example	`infra/cloudformation/pipeline/pipeline-params.example.json`

---

7. Implementation status

Area	Status
Next.js UI (grid, record / pause / stop, countdown)	Done

Event capture (normalized coords, sampling, drag rect, scroll, chaos mode)	Done
Backend: `POST /api/sessions`, list, get (presigned URL), delete	Done
Results: `sessionStorage` handoff + API + S3 via **My Recordings**	Done
Heatmap / Replay / Details	Done
Cognito auth, **My Recordings**, admin flows (as implemented)	Done
CloudFormation parent + nested stacks + infra docs	Done
Terraform mirror + docs	Done
Terraform CI pipeline	Documented in `infra/terraform/README.md`

---

8. Configuration (environment)

Copy **`.env.example`** → **`.env.local`** (gitignored). Key **`NEXT_PUBLIC_*`** variables:

Variable	Notes
`NEXT_PUBLIC_API_URL`	HTTP API base URL; **no trailing slash** (client builds `/api/...`).
`NEXT_PUBLIC_COGNITO_USER_POOL_ID`	From stack / Terraform output.
`NEXT_PUBLIC_COGNITO_CLIENT_ID`	App client id.
`NEXT_PUBLIC_COGNITO_DOMAIN`	Typically `PREFIX.auth.REGION.amazoncognito.com`.
`NEXT_PUBLIC_COGNITO_REDIRECT_URI`	Must match Cognito app client settings (e.g. `https://your-domain/auth/callback` for production).

See **`.env.example`** in the repo root for the canonical list.

---

9. Build & publish (frontend)

```
npm ci          # or npm install
npm run build   # produces out/
```

Smoke-test static output locally:

```
npm run preview   # serves out/ on port 3000 (uses npx serve)
```

Publish to the **site bucket** from stack outputs, then invalidate CloudFront (example names — use **your** bucket and distribution):

```
export AWS_PROFILE=your-profile
aws s3 sync out/ "s3://YOUR_SITE_BUCKET/" --delete
```

```
aws cloudfront create-invalidation --distribution-id YOUR_DIST_ID --paths "/*"
```

Build-time note: `NEXT_PUBLIC_*` is baked into the static bundle at `npm run build`. For production auth, ensure **redirect URIs** and Cognito **callback URLs** match the URL users actually use.

---

10. Deploy on AWS (summary)

10.1 Prerequisites

- AWS account, **AWS CLI v2**, authenticated session (e.g. `aws sso login --profile ...`).
- **Node.js** for local builds.
- Globally unique **Cognito domain prefix** and (for CloudFormation packaging) a **packaging bucket** name.

10.2 CloudFormation — one-command style (from repo root)

```
export AWS_PROFILE=<profile>
export AWS_REGION=us-east-1
export PACKAGING_BUCKET=<unique-artifacts-bucket>
export COGNITO_DOMAIN_PREFIX=<unique-prefix>
# optional: export ENV=dev   # → stack heatfx-dev

./scripts/deploy-aws.sh
```

PowerShell equivalent: `scripts/deploy-aws.ps1`.

After the stack exists: `npm run build`, `aws s3 sync`, **CloudFront invalidation** as in §9. Tighten **CORS** and Cognito **callback / logout URLs** once the CloudFront HTTPS URL is known.

10.3 Optional CodePipeline (CloudFormation path)

1. Create **CodeConnections** (GitHub) → copy **ConnectionArn**.
2. Copy `pipeline-params.example.json` → `pipeline-params.json` (gitignored), fill repository, branch, buckets, Cognito prefix, callbacks.
3. `./scripts/deploy-pipeline.sh` (or `.ps1`).
4. Push to the configured branch or **Release change** in the console.

10.4 Terraform path

Full clone → variables → CloudFront checklist: `infra/terraform/README.md`. Pipeline path uses `buildspec.terraform.yml` (Terraform apply → `npm run build` → `s3 sync` → invalidation).

10.5 Tear-down

Delete the **app** parent stack; empty S3 buckets if deletion stalls. Delete the **pipeline** stack separately if created.

---

11. Security & hygiene

- Never commit `.env.local`, raw access keys, or session tokens.
- Prefer **IAM roles / SSO** for local and CI access.
- Restrict API **CORS** and Cognito URLs to known **https** origins in production.
- Review **IAM** scope for Lambda, CodeBuild, and pipeline roles in your account.

---

12. Current limitations & production notes

- `sessionStorage` is a convenience for in-tab navigation, not durable storage.
- Validate **Cognito redirect URIs**, **CORS**, and **CloudFront** caching behavior after each deploy.
- AWS **pricing and free tier** change — avoid absolute "always free" claims in external marketing; use cautious language (see `SPEC.md` §8 for public-site guardrails).

---

13. References & deep links

Resource	Path / URL
GitHub	https://github.com/OrieBound/heatfx-serverless
Product & technical spec	`docs/SPEC.md`
This overview (Markdown)	`docs/project-overview-heatfx-serverless.md`
Root README	`README.md`
Infra guide	`infra/README.md`
Terraform deploy guide	`infra/terraform/README.md`
Architecture image	`docs/heatfx-serverless-architecture.jpg`
Pipeline template (CFN)	`infra/cloudformation/pipeline/pipeline.yaml`
Build specs	`buildspec.yml`, `buildspec.terraform.yml`

End of project overview. Generate PDF from this file with Pandoc, `md-to-pdf`, or any Markdown → PDF workflow you prefer.